

```
#encoding: UTF-8
#!/usr/bin/env python2
import cv2 as cv
import os
import numpy as np
import time
from opencv_yolo import yolo
import math
import rospy
from geometry_msgs.msg import Twist

from VideoCapture import FastVideoCapture

from sensor_msgs.msg import Range

#超声波数据获取的类
class sonorRange(object):
    def __init__(self):
        super(sonorRange, self).__init__()
        self.initialize_sonor()
        self.openSonor()
        self.subSonorRange()

    #初始化超声波
    def initialize_sonor(self):
        self.sonor_range = 0

    #打开超声波
    def openSonor(self):
        os.system("python openSonor.py")

    #订阅超声波
    def subSonorRange(self):
        rospy.Subscriber("sonor_range", Range, self.callback)

    #回调函数
    def callback(self, data):
        self.sonor_range = data.range

    #获取超声波实时数据
    def getSonorRange(self):
        return self.sonor_range
```

#人体跟随类

class fpdemo(object):

def __init__(self):

super(fpdemo, self).__init__()

self.initialize_ros.py()

self.initialize_vision()

#初始化视觉

def initialize_vision(self):

self.OBJ_THRESH = 0.35

self.NMS_THRESH = 0.05

self.CLASSES = (["person"])

self.yolo = yolo(self.OBJ_THRESH, self.NMS_THRESH, self.CLASSES)

cap_num = 0

self.cap = FastVideoCapture(cap_num)

self.cv_rgb_image = None

self.person_box = None

self.new_boxes = None

self.new_scores = None

self.new_classes = None

self.IMAGE_SIZE = 640

self.IMAGE_SIZE_SMALL = 128

self.IMAGE_WIDTH = 640

self.IMAGE_HEIGHT = 480

self.c_x, self.c_y = self.IMAGE_WIDTH/2, self.IMAGE_HEIGHT/2

#视频显示

def show_image(self, img):

cv.imshow("figure", img)

cv.waitKey(50)

#过滤掉不是人体的识别结果

def filterYoloResults(self, boxes, classes, scores):

new_boxes = []

new_classes = []

new_scores = []

if boxes is not None:

```

for box, score, cl in zip(boxes, scores, classes):
    if cl == 0 and score > self.OBJ_THRESH:
        if len(new_boxes) == 0:
            new_boxes = np.array([box])
            new_scores = np.array([score])
            new_classes = np.array([cl])
        else:
            new_boxes = np.vstack([new_boxes, box])
            new_scores = np.vstack([new_scores, score])
            new_classes = np.vstack([new_classes, cl])

    if len(new_boxes) > 0:
        self.new_boxes = new_boxes*5
        self.new_classes = new_classes
        self.new_scores = new_scores
        return new_boxes*5, new_classes, new_scores
    else:
        return None, None, None
else:
    return None, None, None

```

#人体识别

def detectPerson(self, img):

```

    is_find = False
    # print("w/h: ",img.shape)
    img_ori = img
    img_ori = self.transform_frame(img) #640*640
    img128 = self.transform_frame_128(img) #128*128

```

t1 = time.time()

#加载模型

net = cv.dnn.readNetFromONNX("person128.onnx")

#输入数据处理

```

blob = cv.dnn.blobFromImage(img128, 1 / 255.0, (self.IMAGE_SIZE_SMALL,
self.IMAGE_SIZE_SMALL), [0, 0, 0], swapRB=True, crop=False)
net.setInput(blob)

```

#推理

outputs = net.forward(net.getUnconnectedOutLayersNames())[0]

```

#获得识别结果
boxes, classes, scores = self.yolo.yolov5_post_process_simple(outputs)
t2 = time.time()
# print(t2-t1)

new_boxes, new_classes, new_scores = self.filterYoloResults(boxes, classes, scores)

#物体边框处理
if new_boxes is not None:

    self.yolo.draw(img_ori, new_boxes, new_scores, new_classes)
    is_find = True

#视频显示
self.show_image(img_ori)
return is_find

#图像处理，适配人体识别
def transform_frame(self, frame):
    frame, ratio, (dw, dh) = self.yolo.letterbox(frame, (self.IMAGE_SIZE, self.IMAGE_SIZE))

    return frame

#图像处理，适配人体识别
def transform_frame_128(self, frame):
    frame, ratio, (dw, dh) = self.yolo.letterbox(frame, (self.IMAGE_SIZE_SMALL,
self.IMAGE_SIZE_SMALL))

    return frame

#跟随，速度规划
def follow(self, sonor_range):
    x, y, xsize, ysize = self.calcPersonPositon(self.person_box)

    print(x, y, xsize)

    move_cmd = Twist()
    if sonor_range > 0.95:
        move_cmd.linear.x = 0.25
    elif sonor_range > 0.7:
        move_cmd.linear.x = 0.2

```

```

elif sonor_range > 0.5:
    move_cmd.linear.x = 0.15
else:
    move_cmd.linear.x = 0

if xsize > self.IMAGE_WIDTH*0.5:
    move_cmd.linear.x = 0

if abs(x) > 1:
    move_cmd.angular.z = x/self.IMAGE_WIDTH*2
self.pub.publish(move_cmd)

#获得正前方的人体方框
def getPersonBox(self):
    self.resetPersonDepthBox()
    if self.new_boxes is not None:
        for new_box in self.new_boxes:
            left, top, right, bottom = new_box
            top = int(top)
            left = int(left)
            right = int(right)
            bottom = int(bottom)
            self.calcCenterPersonBox(new_box)

#计算得出正前方的方框
def calcCenterPersonBox(self, new_box):
    if self.person_box is None:
        self.person_box = new_box
    else:
        x, y, x_size_p, y_size_p = self.calcPersonPositon(new_box)
        delta_x_newbox = abs(x - self.c_x)
        x, y, x_size_p, y_size_p = self.calcPersonPositon(self.person_box)
        delta_x_box = abs(x - self.c_x)
        if delta_x_box > delta_x_newbox:
            self.person_box = new_box

    return self.person_box

#复位人体方框信息
def resetPersonDepthBox(self):
    self.person_box = None

```

#计算人体的坐标和尺寸

def calcPersonPositon(self, box):

left, top, right, bottom = box

px = left + right

py = bottom + top

px = px*0.5

py = py*0.5

x = self.c_x - px

y = self.c_y - py

x_size_p = abs(left - right)

y_size_p = abs(top - bottom)

return x, y, x_size_p, y_size_p

#初始化 ros

def initialize_rospy(self):

rospy.init_node('fpdemo', anonymous=True)

self.pub = rospy.Publisher('cmd_vel', Twist, queue_size=1)

#显示彩色图

def showCvRgbImage(self):

cv.imshow("", self.cv_rgb_image)

cv.waitKey(50)

#获取彩色图

def getRgbImage(self):

self.cv_rgb_image = self.cap.read()

#主函数

def run(self):

sr = sonorRange()

while not rospy.is_shutdown():

self.getRgbImage()

sonor_range = sr.getSonorRange()

print("sonor range: ", sonor_range)

if self.detectPerson(self.cv_rgb_image):

print("find a man.")

```
self.getPersonBox()  
self.follow(sonor_range)
```

```
rospy.spin()
```

```
if __name__ == "__main__":  
    fpdemo = fpdemo()  
    fpdemo.run()
```